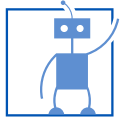


Hinweis zum Urheberrecht

Diese Aufzeichnung und ihre Inhalte sind Eigentum des Lehrstuhls für Robotik, Künstliche Intelligenz und Echtzeitsysteme der Technischen Universität München und werden ausschließlich für angemeldete Studierende des Moduls Kognitive Systeme (IN2222) als Videostream auf der Lernplattform Moodle bereitgestellt. **Download und Weitergabe sind weder ganz noch in Teilen gestattet.**

Copyright Notice

This recording and its contents are property of the Chair of Robotics, Artificial Intelligence and Real-Time Systems of the Technical University of Munich and are provided exclusively for registered students of the module Cognitive Systems (IN2222) as a video stream on the learning platform Moodle. **Download and distribution are prohibited in whole or in part.**



Robotics, Artificial Intelligence
and Real-time Systems



Reinforcement Learning: Monte Carlo Methods

Prof. Dr.-Ing. habil. Alois Knoll
Kejia Chen, M.Sc.

MC Prediction

- In practical cases, the MDP is usually unknown ($\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$).
- Monte-Carlo-based approaches don't require knowledge of the MDP as they learn from sampled state trajectories:



- **Monte Carlo prediction:** Return is calculated for all states in each sampled trajectory. For the evaluation of value functions, experienced returns are averaged.

π	policy	R	stochastic reward function
A	stochastic action function	S	stochastic state function
q	value for a specific action	T	terminal state

[§]Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, p.92.
Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann.

MC Prediction: Pseudo Code

- 1: Input: a policy π to be evaluated
- 2: Initialize:
- 3: $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
- 4: $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$
- 5: Loop forever (for each episode):
- 6: Generate an episode following $\pi : S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
- 7: $G \leftarrow 0$
- 8: Loop for each step of episode, $t = T - 1, T - 2, \dots, 0$:
- 9: $G \leftarrow \gamma G + R_{t+1}$
- 10: Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$: ——— first visit to S_t
- 11: Append G to $Returns(S_t)$
- 12: $V(S_t) \leftarrow \text{average}(Returns(S_t))$

Incremental implementation:
 $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$

γ	discount rate
π	policy
A	stochastic action function
G	return
q	value for a specific action
R	stochastic reward function
S	stochastic state function
s	particular state
V	value estimates
α	learning rate

[§]Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, **p.92**.
 Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann

MC Control

- Without model, state values alone are not sufficient to determine a policy. One of primary goals for Monte Carlo methods is to estimate **state-action values**.
- Exploring starts: If π is a deterministic policy, many state-action pairs may never be visited.



- Monte Carlo control:** updating value and policy for every sample:

$$\pi_0 \xrightarrow{\text{eval.}} q_{\pi_0} \xrightarrow{\text{impr.}} \pi_1 \xrightarrow{\text{eval.}} q_{\pi_1} \xrightarrow{\text{impr.}} \pi_2 \dots \pi_* \xrightarrow{\text{eval.}} q_*$$

π	policy	R	stochastic reward function
A	stochastic action function	S	stochastic state function
q	value for a specific action	T	terminal state

[§]Sutton, R. S. and Barto, A. G. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, **p.92,97**.
Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann

MC Control: With Exploring Starts

- 1: Initialize:
- 2: $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
- 3: $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- 4: $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- 5:
- 6: Loop forever (for each episode):
- 7: Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0 — Exploring starts
- 8: Generate an episode from S_0, A_0 , following $\pi : S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
- 9: $G \leftarrow 0$
- 10: Loop for each step of episode, $t = T - 1, T - 2, \dots, 0$:
- 11: $G \leftarrow \gamma G + R_{t+1}$
- 12: Unless the pair S_t, A_t Appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:
- 13: Append G to $Returns(S_t, A_t)$
- 14: $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ — Estimate state-action values
- 15: $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$ — greedy

ϵ	probability of random action
γ	discount rate
π	policy
A	stochastic action function
G	return
q	value for a specific action
R	stochastic reward function
S	stochastic state function
s	particular state
V	value estimates

[§]Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, **p.99**.

MC Control: Without Exploring Starts

```

1: Algorithm parameter: small  $\epsilon > 0$ 
2: Initialize:
3:    $\pi \leftarrow$  an arbitrary  $\epsilon$ -soft policy
4:    $Q(s, a) \in \mathbb{R}$  (arbitrarily), for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
5:    $Returns(s, a) \leftarrow$  an empty list, for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
6:
7: Repeat forever (for each episode):
8:   Generate an episode following  $\pi : S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$ 
9:    $G \leftarrow 0$ 
10:  Loop for each step of episode,  $t = T - 1, T - 2, \dots, 0$ :
11:     $G \leftarrow \gamma G + R_{t+1}$ 
12:    Unless the pair  $S_t, A_t$  appears in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$ :
13:      Append  $G$  to  $Returns(S_t, A_t)$ 
14:       $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ 
15:       $A^* \leftarrow \arg \max_a Q(S_t, a)$  (with ties broken arbitrarily)
16:      For all  $a \in \mathcal{A}(S_t)$ :
17:         $\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$ 

```

ϵ	probability of random action
γ	discount rate
π	policy
A	stochastic action function
G	return
q	value for a specific action
R	stochastic reward function
S	stochastic state function
s	particular state
V	value estimates

[§]Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. 2nd ed. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2018, **p.101**. Slide is adapted from CogSys21 RL lecture held by Prof. Dr. Ing. Noah Klarmann

ϵ —greedy: Keeping a non-zero probability for every action. This ensures an exhaustive exploration of all states to guarantee convergence towards an optimal policy.